23CSE202 DATABASE MANAGEMENT SYSTEMS
S3 B.Tech CSE
Department of Computer Science and Engineering
Amrita School of Computing, Amritapuri Campus

3. Database Design Document

Group No : C14
Team Members :

Roll No	Student Name
AM.SC.U4CSE25224	Jayanth Amanchy
AM.SC.U4CSE25216	Gali Abhilaash
AM.SC.U4CSE25240	Pallapothu Hanumath Sai Krishna
AM.SC.U4CSE25248	Shantanu S Shenoy

Project Title : Online Voting System

3.1. Detailed description of the project

The **Online Voting System** is a database-based application designed to manage the voting process in an organized, secure, and efficient manner. The system stores and manages information related to voters, candidates, elections, votes, and administrators.

A **Voter** registers in the system using their personal details such as Voter ID, name, age, email address, username, and password. Each voter is uniquely identified using a Voter ID. Only registered voters are allowed to participate in an election.

A **Candidate** is a person who participates in an election. The system stores details such as Candidate ID, name, age, party name, and other relevant information. Multiple candidates can participate in an election.

An **Election** represents a voting event conducted by the system. The system stores information such as Election ID, Election Name, Election Date, Start Date, End Date, and Status. An election can have multiple candidates, and candidates may participate in different elections.

The **Vote** entity records the voting activity. When a voter casts a vote, the system stores the Vote ID along with the corresponding Voter ID, Candidate ID, and Election ID. The system ensures that a voter can cast only **one vote in a particular election**.

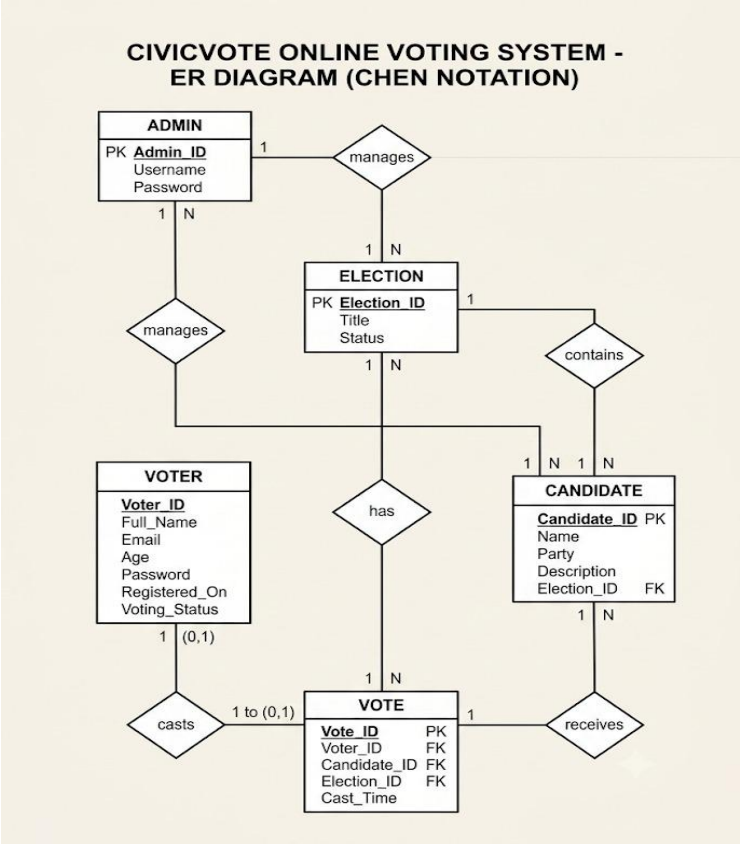
The **Admin** manages the overall system. The administrator can add, update, and delete candidate details, manage voter information, create elections, and view election results.

The main users of the system are **Voters and Administrators**. Voters can register, log in, view candidates, cast their votes, and view results. Administrators manage voters, candidates, and elections.

Business Rules

1. Every voter must have a unique Voter ID.
2. Every candidate must have a unique Candidate ID.
3. Every election must have a unique Election ID.
4. A voter must be registered before casting a vote.
5. A voter can vote only once in a particular election.
6. Each vote must be associated with one voter, one candidate, and one election.
7. An election can have multiple candidates.
8. A candidate may participate in multiple elections.
9. Only administrators are authorized to manage candidates and elections.
10. Election results are calculated based on the votes received by each candidate.

3.2. ER Diagram



3.3 Relational Schema

The following relational schema is derived from the ER diagram in Section 3.2. It lists the five relations of the Online Voting System together with their primary keys, foreign keys, and integrity constraints.

VOTER

Attribute	Key / Constraint
Voter_ID	PRIMARY KEY, NOT NULL, UNIQUE
Name	NOT NULL
Age	NOT NULL, CHECK (Age >= 18)
Email	NOT NULL, UNIQUE
Password	NOT NULL

ELECTION

Attribute	Key / Constraint
Election_ID	PRIMARY KEY, NOT NULL, UNIQUE
Election_Name	NOT NULL, UNIQUE
Election_Date	NOT NULL
Status	NOT NULL, CHECK (Status IN ('UPCOMING','ONGOING','CLOSED'))

CANDIDATE

Attribute	Key / Constraint
Candidate_ID	PRIMARY KEY, NOT NULL, UNIQUE
Candidate_Name	NOT NULL
Party	NOT NULL
Election_ID	NOT NULL, FOREIGN KEY → Election(Election_ID)

VOTE

Attribute	Key / Constraint
Vote_ID	PRIMARY KEY, NOT NULL, UNIQUE
Voter_ID	NOT NULL, FOREIGN KEY → Voter(Voter_ID)

Candidate_ID	NOT NULL, FOREIGN KEY → Candidate(Candidate_ID)
Election_ID	NOT NULL, FOREIGN KEY → Election(Election_ID)
Voter_ID + Election_ID	COMPOSITE UNIQUE KEY, NOT NULL

ADMIN

Attribute	**Key / Constraint**
Admin_ID	PRIMARY KEY, NOT NULL, UNIQUE
Admin_Name	NOT NULL
Email	NOT NULL, UNIQUE
Password	NOT NULL

3.4 Normalization and Functional dependency analysis

1. Functional Dependency (FD) Identification & Significance

We identified the functional dependencies (FDs) by looking at how our online voting system is supposed to work in real life. These FDs act as rules to keep our data accurate and consistent.

* VOTER Relation

* FD1: Voter_ID -> {Name, Age, Email, Password}

* Significance: This ensures that every voter has a unique ID, and that ID brings up exactly one specific person's record.

* FD2: Email -> {Voter_ID, Name, Age, Password}

* Significance: This enforces our business rule that two people cannot sign up using the exact same email address, preventing duplicate profiles.

* ELECTION Relation

* FD1: Election_ID -> {Election_Name, Election_Date, Status}

* Significance: Each election needs its own distinct timeline and operational state, like UPCOMING, ONGOING, or CLOSED.

* FD2: Election_Name -> {Election_ID, Election_Date, Status}

* Significance: This stops us from accidentally creating two different elections with the exact same title.

* CANDIDATE Relation

* FD1: Candidate_ID -> {Candidate_Name, Party, Election_ID}

* Significance: This tracks each candidate uniquely, mapping them to their political party and making sure they only run in one specific election.

* VOTE Relation

* FD1: Vote_ID -> {Voter_ID, Candidate_ID, Election_ID}

* Significance: This lets us audit the system because every cast ballot gets a completely unique receipt ID.

* FD2: (Voter_ID, Election_ID) -> {Vote_ID, Candidate_ID}

* Significance: This is a huge rule for our project. By making the combination of Voter_ID and Election_ID point to the vote details, the database physically stops a voter from casting more than one vote in the same election.

* ADMIN Relation

* FD1: Admin_ID -> {Admin_Name, Email, Password}

* Significance: Uniquely identifies our system administrators for secure access.

* FD2: Email -> {Admin_ID, Admin_Name, Password}

* Significance: Just like the voters, admins must have unique email logins so credentials don't overlap.

2. Candidate Key Selection & Attribute Closure Proofs

To find our candidate keys, we calculated the attribute closures to make sure that the chosen keys can find every other attribute in that table.

* VOTER Relation

  * {Voter_ID}+ = {Voter_ID, Name, Age, Email, Password} (Using FD1, we can find everything)

  * {Email}+ = {Email, Voter_ID, Name, Age, Password} (Using FD2, we can also find everything)

  * Candidate Keys: Since both closures give us the full set of attributes, our candidate keys are Voter_ID and Email. We chose Voter_ID as the Primary Key.

* ELECTION Relation

  * {Election_ID}+ = {Election_ID, Election_Name, Election_Date, Status} (By FD1)

  * {Election_Name}+ = {Election_Name, Election_ID, Election_Date, Status} (By FD2)

  * Candidate Keys: Election_ID and Election_Name are both valid candidate keys. We are using Election_ID as the Primary Key.

* CANDIDATE Relation

  * {Candidate_ID}+ = {Candidate_ID, Candidate_Name, Party, Election_ID} (By FD1)

  * Candidate Key: Candidate_ID is our only candidate key, so it becomes the Primary Key.

* VOTE Relation

  * {Vote_ID}+ = {Vote_ID, Voter_ID, Candidate_ID, Election_ID} (By FD1)

  * {Voter_ID, Election_ID}+ = {Voter_ID, Election_ID, Vote_ID, Candidate_ID} (By FD2)

  * Candidate Keys: We have two candidate keys here. Vote_ID is a single attribute key, and (Voter_ID, Election_ID) is a composite candidate key. We chose Vote_ID as the Primary Key.

* ADMIN Relation

  * {Admin_ID}+ = {Admin_ID, Admin_Name, Email, Password} (By FD1)

* {Email}+ = {Email, Admin_ID, Admin_Name, Password} (By FD2)

* Candidate Keys: Both Admin_ID and Email can resolve the full table, making them our candidate keys. Admin_ID is selected as the Primary Key.

3. Step-by-Step Normalization Process (1NF to BCNF)

We tested our tables against the normal forms we learned about to make sure we won't run into any weird data anomalies when inserting, updating, or deleting records.

* First Normal Form (1NF)

* What we checked: Every value in our tables must be atomic, meaning no lists or multiple values mixed into a single cell.

* Our justification: Looking at fields like Name, Status, and Party, everything holds just a single piece of data. There are no repeating groups, so our whole design is easily in 1NF.

* Second Normal Form (2NF)

* What we checked: The database must be in 1NF and have zero partial dependencies. A non-key attribute cannot depend on just a piece of a composite candidate key.

* Our justification: For the VOTER, ELECTION, CANDIDATE, and ADMIN tables, the candidate keys are single attributes anyway. You can't have a partial dependency if the key can't be broken down into pieces, so they pass. For the VOTE table, our composite key is (Voter_ID, Election_ID). The non-key fields depend on the entire combination to show who voted where, not just one or the other. Therefore, everything is in 2NF.

* Third Normal Form (3NF)

* What we checked: The database must be in 2NF and have no transitive dependencies, meaning non-key fields shouldn't determine other non-key fields.

* Our justification: Every single non-prime attribute in our design relies directly on a candidate key. We don't have any scenarios where Field A

determines Field B, which then determines Field C. Since there are no middle-man dependencies, the schema satisfies 3NF.

* Boyce-Codd Normal Form (BCNF)

  * What we checked: BCNF is stricter than 3NF. It requires that for every functional dependency X -> Y, the left side (X) must be a superkey.

  * Our justification: We went back and analyzed all of our original FDs. For VOTER, ELECTION, VOTE, and ADMIN, the left-hand sides—like Voter_ID, Email, Election_ID, Election_Name, Vote_ID, and the composite (Voter_ID, Election_ID)—are all verified candidate keys. For the CANDIDATE table, Candidate_ID is also a candidate key. Because every single determinant in our project is a superkey, our design successfully hits BCNF.